

Rapport de Travaux Pratiques

Bases de Données Avancées

Synthèse complète : TP1 à TP8

Réalisé par : Meriem ER.RAHMOUNI

Encadré par : M. Hamid HRIMECH

TP 1 : Vues

Introduction

Ce rapport présente les travaux pratiques réalisés dans le cadre du cours de **Bases de Données Avancées**. L'objectif de ce TP est de manipuler les concepts de **Vues** (tables virtuelles encapsulant des requêtes de sélection complexes) et d'**Index** afin d'appréhender les problématiques d'abstraction, de sécurité, d'intégrité et d'optimisation des performances au sein d'un Système de Gestion de Bases de Données (SGBD).

Schéma Relationnel de Référence

Les exercices s'appuient sur le schéma relationnel suivant :

- **EMP** (Matr, NomE, Poste, DatEmb, Sup, Salaire, Commission, #NumDept)
- **DEPT** (NumDept, NomDept, Lieu)
- **PROJET** (CodeP, NomP)
- **PARTICIPATION** (#Matr, #CodeP, Fonction)

Exercice 1 : Les Vues

Question 1 : Création de la vue composite V_EMP

Consigne

Créez une vue V_EMP contenant : le matricule, le nom, le numéro de département, la somme de la commission et du salaire baptisée GAINS, le lieu du département.

Solution

```
1 CREATE VIEW V_EMP AS
2 SELECT
3     e.Matr ,
4     e.NomE ,
5     e.NumDept ,
6     (NVL(e.Salaire , 0) + NVL(e.Commission , 0)) AS GAINS ,
7     d.Lieu
8 FROM EMP e
9 JOIN DEPT d ON e.NumDept = d.NumDept ;
```

1 – Création de la vue V_EMP

Explication

La vue V_EMP est une vue complexe car elle encapsule une jointure entre les tables EMP et DEPT. De plus, la fonction NVL (ou COALESCE) est recommandée pour traiter les valeurs nulles éventuelles dans le calcul arithmétique de la colonne calculée GAINS.

Question 2 : Sélection avec condition sur colonne calculée

Consigne

Sélectionnez les lignes de V_EMP dont le salaire total est supérieur à 10 000 F.

Solution

```
1 SELECT * FROM V_EMP
2 WHERE GAINS > 10000 ;
```

2 – Sélection avec filtrage sur la vue

Explication

Bien que **GAINS** soit le produit d'une expression de calcul dynamique sous-jacente, il est possible de filtrer directement dessus à l'aide de la clause **WHERE** comme s'il s'agissait d'une colonne physique ordinaire.

Question 3 : Mise à jour à travers une vue composite

Consigne

Essayez de mettre à jour le nom de l'employé **MARTIN** à travers la vue **V_EMP**.

Solution

```
1 UPDATE V_EMP
2 SET NomE = 'MARTIN_MODIF'
3 WHERE NomE = 'MARTIN';
```

3 – Tentative de mise à jour du nom de l'employé

Explication

Selon les règles du SGBD (ex : Oracle), une vue basée sur une simple jointure permet la mise à jour des colonnes de la *table clé-préservée* (ici la table **EMP**, car la relation est de N à 1 vers **DEPT**). La mise à jour du nom de l'employé sera donc **acceptée** et répercutée directement sur la table physique **EMP**.

Question 4 : Vue restrictive et comportement sans CHECK OPTION

Consigne

Créez une vue **VEMP10** qui ne contienne que les employés du département 10 de la table **EMP** (n'utilisez pas l'option **CHECK** pour cette création). Insérez dans cette vue un employé **SOULIER** qui appartient au département 20. Essayez ensuite de retrouver cet employé au moyen de la vue **VEMP10** puis au moyen de la table **EMP**.

Solution

```
1 -- Cr ation de la vue sans l'option CHECK
2 CREATE VIEW VEMP10 AS
3 SELECT * FROM EMP
4 WHERE NumDept = 10;
5
6 -- Insertion d'un employé dans le d partement 20
```

```

7 INSERT INTO VEMP10 (Matr, NomE, NumDept, Salaire)
8 VALUES (999, 'SOULIER', 20, 5000);
9
10 -- V r i f i c a t i o n   v i a   l a   v u e
11 SELECT * FROM VEMP10 WHERE NomE = 'SOULIER';
12
13 -- V r i f i c a t i o n   v i a   l a   t a b l e
14 SELECT * FROM EMP WHERE NomE = 'SOULIER';

```

4 – Création sans restriction et insertion

Explication

L'insertion de l'employé SOULIER (département 20) via la vue est **acceptée** par le SGBD. Cependant, la ligne disparaît immédiatement de la vue VEMP10 car elle ne valide pas la condition WHERE NumDept = 10. En revanche, la ligne est bel et bien présente dans la table physique EMP.

Question 5 : Destruction et recréation avec WITH CHECK OPTION

Consigne

Détruisez cette vue VEMP10 et recréez-la avec l'option CHECK.

Solution

```

1 DROP VIEW VEMP10;
2
3 CREATE VIEW VEMP10 AS
4 SELECT * FROM EMP
5 WHERE NumDept = 10
6 WITH CHECK OPTION;

```

5 – Recréation de la vue avec WITH CHECK OPTION

Explication

La clause WITH CHECK OPTION agit comme une contrainte d'intégrité dynamique. Elle empêche toute insertion ou modification de données via la vue qui produirait des lignes invisibles pour cette même vue.

Question 6 : Test de violation de la contrainte CHECK OPTION

Consigne

Essayez d'insérer un employé BALARD pour le département 30. Que se passe-t-il ? Essayez de modifier le département d'un employé visualisé à l'aide de cette vue.

Solution

```

1  -- Tentative d'insertion hors p r i m t r e
2  INSERT INTO VEMP10 (Matr, NomE, NumDept)
3  VALUES (888, 'BALARD', 30);
4  -- R s u l t a t   a t t e n d u   :   E r r e u r   O R A - 0 1 4 0 2 :   v i e w   W I T H   C H E C K   O P T I O N
      w h e r e - c l a u s e   v i o l a t i o n
5
6  -- Tentative de mise      j o u r   d u   d e p a r t e m e n t   d ' u n   e m p l o y é   d e
      l a   v u e
7  UPDATE VEMP10
8  SET NumDept = 30
9  WHERE NomE = 'MARTIN_DEPT10';
10 -- R s u l t a t   a t t e n d u   :   R E F U S   ( M m e   e r r e u r   d e   v i o l a t i o n   d e
      c o n t r a i n t e )

```

6 – Tests d'écritures restrictives

Explication

Toutes les opérations violant la clause `WHERE NumDept = 10` sont systématiquement **refusées** par le SGBD, assurant ainsi l'étanchéité des modifications permises par la vue.

Question 7 : Calculs de pourcentages et sous-requêtes inline

Consigne

Liste des salaires des employés avec le pourcentage par rapport au total des salaires de leur département (utilisez une vue qui fournira le total des salaires). Vous pourrez chercher une autre solution qui n'utilise pas de vue mais un select emboîté dans le from.

Solution

Méthode A : Utilisation d'une vue intermédiaire

```

1  -- 1. C r a t i o n   d e   l a   v u e   d e s   t o t a u x   p a r   d e p a r t e m e n t
2  CREATE VIEW V_TOT_SAL AS
3  SELECT NumDept, SUM(Salaire) AS TotalSalDept
4  FROM EMP
5  GROUP BY NumDept;

```

```

6
7 -- 2. Requête finale réalisant la jointure
8 SELECT
9     e.NomE,
10    e.NumDept,
11    e.Salaire,
12    ROUND((e.Salaire / v.TotalSalDept) * 100, 2) AS Pourcentage
13 FROM EMP e
14 JOIN V_TOT_SAL v ON e.NumDept = v.NumDept;

```

7 – Solution avec création de vue d'agrégation

Méthode B : Solution optimisée sans vue (Sous-requête / Table en ligne)

```

1 SELECT
2     e.NomE,
3     e.NumDept,
4     e.Salaire,
5     ROUND((e.Salaire / t.TotalSalDept) * 100, 2) AS Pourcentage
6 FROM EMP e
7 JOIN (
8     SELECT NumDept, SUM(Salaire) AS TotalSalDept
9     FROM EMP
10    GROUP BY NumDept
11 ) t ON e.NumDept = t.NumDept;

```

8 – Solution avec Select emboîté dans le FROM

Explication

La méthode B (sous-requête imbriquée dans la clause **FROM**) évite la persistance d'un objet de vue dans le dictionnaire de la base de données. Elle calcule de façon dynamique et temporaire la table d'agrégation pour réaliser la jointure à la volée.

Synthèse comparative des concepts étudiés

À l'instar des conclusions menées lors des études de performances, voici un tableau récapitulatif de l'utilité et de la modifiabilité des objets manipulés au cours de ce TP :

TABLE 1 – Propriétés et comportement des structures SQL

Mécanisme	Type d'Objet	Modifiable (DML)	Impact Performance
Vue Simple	Virtuel	Oui (sous conditions)	Neutre (exécution dynamique)
Vue d'Agrégation	Virtuel	Non	Élevé (recalcul total)
WITH CHECK OPTION	Sécurité / Intégrité	Oui (si valide le WHERE)	Léger contrôle à l'écriture

Conclusion

Ce travail pratique a permis de valider les compétences fondamentales liées à l'abstraction de données et à l'indexation. La maîtrise des vues, enrichie des options restrictives telles que `WITH CHECK OPTION`, offre des solutions élégantes pour masquer la complexité structurelle des tables physiques et isoler les environnements selon les droits des utilisateurs. Par ailleurs.

TP 2 : Les Vues

Introduction

Ce rapport présente les travaux pratiques réalisés sur le thème des **Vues SQL**. Une vue est une table virtuelle dont le contenu est défini par une requête SQL. Elle offre une couche d'abstraction au-dessus des tables physiques, facilitant ainsi la sécurité, la confidentialité et la maintenabilité des données.

Schéma de la base de données employés

Les exercices s'appuient sur l'implémentation du modèle relationnel suivant :

```
1 CREATE TABLE Departement (  
2     dept_id NUMBER PRIMARY KEY,  
3     nom_dept VARCHAR2(50),  
4     ville VARCHAR2(50)  
5 );  
6  
7 CREATE TABLE Employe (  
8     emp_id NUMBER PRIMARY KEY,  
9     nom VARCHAR2(50),  
10    salaire NUMBER(8,2),  
11    dept_id NUMBER,  
12    FOREIGN KEY (dept_id) REFERENCES Departement(dept_id)  
13 );
```

9 – Définition des tables physiques

Exercice 1 : Création d'une Vue Simple

Consigne

1. Créer une vue `emp_paris` affichant les employés travaillant dans un département situé à Paris.
2. Afficher le contenu de la vue et tester un filtrage supplémentaire.

Solution

```
1  -- 1. Creation de la vue simple
2  CREATE VIEW emp_paris AS
3  SELECT e.emp_id, e.nom, e.salaire
4  FROM Employe e
5  JOIN Departement d ON e.dept_id = d.dept_id
6  WHERE d.ville = 'Paris';
7
8  -- 2. Affichage du contenu global
9  SELECT * FROM emp_paris;
10
11 -- Filtrage supplementaire sur le salaire
12 SELECT * FROM emp_paris WHERE salaire > 3000;
```

10 – Création et interrogation de la vue `emp_paris`

Explication

La vue `emp_paris` encapsule la jointure entre `Employe` et `Departement` ainsi que la clause restrictive sur la ville. L'utilisateur final interroge la vue de manière transparente, sans avoir besoin de réécrire la jointure. La vue ne stocke aucune donnée physique ; la requête sous-jacente est exécutée dynamiquement lors de chaque appel.

Exercice 2 : Vue Complexe avec Agrégations

Consigne

Créer une vue `stat_dept` affichant pour chaque département son nom, le salaire minimum, le salaire maximum et le salaire moyen.

Solution

```
1 CREATE VIEW stat_dept AS
2 SELECT d.nom_dept,
3        MIN(e.salaire) AS salaire_min,
4        MAX(e.salaire) AS salaire_max,
5        AVG(e.salaire) AS salaire_moyen
6 FROM Employe e
7 JOIN Departement d ON e.dept_id = d.dept_id
8 GROUP BY d.nom_dept;
9
10 -- Utilisation de la vue avec filtrage conditionnel
11 SELECT * FROM stat_dept WHERE salaire_moyen > 4000;
```

11 – Création de la vue `stat_dept`

Explication

Il s'agit ici d'une vue complexe car elle utilise des fonctions d'agrégation (`MIN`, `MAX`, `AVG`) associées à une clause `GROUP BY`. En raison de ces regroupements, cette vue est strictement **non modifiable** : aucune opération directe d'insertion ou de mise à jour (`INSERT/UPDATE`) n'est permise sur ses colonnes.

Exercice 3 : L'option WITH CHECK OPTION

Consigne

Créer une vue `emp_lyon` affichant les employés du département situé à Lyon et empêcher toute modification ou insertion de données violant cette condition géographique.

Solution

```
1 CREATE VIEW emp_lyon AS
2 SELECT e.*
3 FROM Employe e
4 JOIN Departement d ON e.dept_id = d.dept_id
5 WHERE d.ville = 'Lyon'
6 WITH CHECK OPTION;
```

12 – Création de la vue avec restriction d'intégrité

Explication

La clause `WITH CHECK OPTION` garantit que toutes les requêtes de mise à jour ou d'insertion (DML) exécutées via la vue respectent scrupuleusement le critère du `WHERE`. Par exemple, si l'on tente d'insérer un employé affecté à une ville différente ou de modifier la ville d'un employé de Lyon vers Paris, le SGBD rejettera l'opération et lèvera une erreur d'intégrité.

Exercice 4 : L'option WITH READ ONLY

Consigne

Créer une vue en lecture seule listant l'ensemble des employés afin d'interdire structurellement toute modification.

Solution

```
1 CREATE VIEW emp_readonly AS
2 SELECT * FROM Employe
3 WITH READ ONLY;
```

13 – Création de la vue en lecture seule

Explication

La clause `WITH READ ONLY` supprime toute possibilité d'exécuter des requêtes d'écriture (`INSERT`, `UPDATE`, `DELETE`) à travers la vue. Si un utilisateur tente de modifier une valeur (ex : `UPDATE emp_readonly SET salaire = 5000 WHERE emp_id = 1;`), le système bloque immédiatement l'action avec un message d'erreur d'écriture (ex : *ORA-42399*). Ce mécanisme est idéal pour l'exposition sécurisée de données historiques ou de rapports analytiques.

Exercice 5 : Sécurité et Abstraction

Travail demandé

1. Créer une vue `emp_public` en masquant la colonne sensible du salaire.
2. Analyser l'impact réel de l'utilisation des vues classiques sur la performance du système.

Solution

```
1 CREATE VIEW emp_public AS
2 SELECT emp_id, nom, dept_id
3 FROM Employe
4 WITH READ ONLY;
```

14 – Création de la vue de masquage de colonne

Analyse & Réflexion : Vues et Performances

La réponse est nuancée selon la nature de la vue employée :

- **Vue classique** : Elle n'apporte **aucun gain de performance** intrinsèque. Elle est réévaluée dynamiquement lors de chaque appel, agissant uniquement comme une macro ou un raccourci de requête.
- **Vue matérialisée** : Elle offre un **gain très important**. Les données calculées sont physiquement stockées sur disque, ce qui évite de rejouer des traitements lourds (jointures ou agrégations) à chaque consultation.

Exercice 6 : Analyse Globale et Réflexion Approfondie

1. Architecture de sécurité basée sur les rôles et les vues

Pour répondre au scénario d'envergure internationale proposé , nous mettons en place un ensemble de vues filtrées, couplées à une distribution de privilèges spécifiques :

```

1  -- Vue Managers : Filtrage dynamique des employ s par
    d p a r t e m e n t
2  CREATE VIEW vue_manager AS
3  SELECT e.emp_id, e.nom, e.salaire, e.dept_id
4  FROM Employe e
5  WHERE e.dept_id = SYS_CONTEXT('DEPT_CTX', 'DEPT_ID');
6
7  -- Vue Employ s : Acc s restreint sans visibilit sur les
    salaires tiers
8  CREATE VIEW vue_employe AS
9  SELECT emp_id, nom, dept_id
10 FROM Employe
11 WITH READ ONLY;
12
13 -- Vue RH : Acc s complet et total aux donn es
14 CREATE VIEW vue_rh AS
15 SELECT * FROM Employe
16 WITH READ ONLY;
17
18 -- Attribution fine des privil ges par r le applicatif
19 GRANT SELECT ON vue_manager TO ROLE_MANAGER;
20 GRANT SELECT ON vue_employe TO ROLE_EMPLOYE;
21 GRANT SELECT ON vue_rh TO ROLE_RH;
22
23 -- R vocation stricte des acc s directs aux tables physiques
24 REVOKE SELECT ON Employe FROM ROLE_MANAGER;
25 REVOKE SELECT ON Employe FROM ROLE_EMPLOYE;

```

15 – Implémentation des profils d'accès sécurisés

2. Limites des vues classiques à grand volume

Dans un contexte de très volumétrie de données (centaines de milliers de lignes), les vues classiques affichent plusieurs limites majeures :

- **Réévaluation complète** : Aucun mécanisme de cache n'étant disponible, les jointures et les filtres sont entièrement réexécutés à chaque sélection.
- **Plans d'exécution lourds** : L'imbrication successive de plusieurs vues génère des

requêtes finales très denses et complexes à optimiser pour le moteur de base de données.

- **Recalcul des agrégations** : Les regroupements (`GROUP BY`) obligent à re-parcourir la totalité de la table physique à chaque consultation.

3. Intérêt et mise en œuvre des vues matérialisées

Les vues matérialisées permettent d'outrepasser ces limites en pré-calculant et en archivant le résultat de la requête.

```

1 CREATE MATERIALIZED VIEW mv_stat_dept
2 REFRESH COMPLETE ON DEMAND AS
3 SELECT d.nom_dept ,
4         MIN(e.salaire) AS salaire_min ,
5         MAX(e.salaire) AS salaire_max ,
6         AVG(e.salaire) AS salaire_moyen
7 FROM Employe e
8 JOIN Departement d ON e.dept_id = d.dept_id
9 GROUP BY d.nom_dept ;

```

16 – Création d'une vue matérialisée statistique

Avantages : Lectures immédiates, idéal pour les outils décisionnels (*Business Intelligence*).

Inconvénients : Risque de désynchronisation des données en attendant le rafraîchissement, et surcoût en espace de stockage sur disque.

4. Comparaison : Vues vs Mechanisms GRANT/REVOKE

TABLE 2 – Matrice comparative de gestion des accès

Critère	Vues	GRANT / REVOKE
Granularité des colonnes	Fine (Sélection explicite)	Limitée selon le SGBD
Filtrage des lignes	Oui (via clause <code>WHERE</code>)	Non
Transformations de données	Oui (Calculs, <code>NVL</code> , Masquage)	Non
Gestion des droits	Indirecte (Sécurise le périmètre)	Directe (Donne l'action)

Recommandation : Combiner l'utilisation des vues pour filtrer la donnée et les commandes `GRANT/REVOKE` pour accorder les droits d'accès à ces vues.

5. Analyse de la sécurité absolue par les vues

Non, une vue ne peut pas garantir à elle seule la sécurité globale et imperméable d'un système informatique. Elle comporte plusieurs vulnérabilités intrinsèques :

1. Risques élevés d'injections SQL si les applications intermédiaires exécutent des requêtes dynamiques mal nettoyées.

2. Possibilité de contournement si les privilèges d'accès direct aux tables physiques sous-jacentes sont mal configurés.
3. Absence totale de chiffrement natif des données stockées sur le disque de stockage.
4. Impossibilité de mener un audit fin des accès en lecture seule.

Il est donc impératif de mettre en place une **stratégie multi-couches** (Vues + Privilèges + Chiffrement + Audit système).

6. Stratégie d'optimisation globale proposée

Pour préserver d'excellentes performances à grande échelle, la configuration technique suivante doit être déployée :

1. **Indexation stratégique** : Créer des index sur les clés de jointures et les critères fréquents de recherche :

```
1 CREATE INDEX idx_employe_dept ON Employe(dept_id);
2 CREATE INDEX idx_dept_ville ON Departement(ville);
```

2. **Partitionnement des données** : Segmenter les tables volumineuses (ex : par liste de pays ou de régions) pour restreindre l'analyse physique des requêtes aux seuls sous-ensembles ciblés.
3. **Indexation sur Vues Matérialisées** : Mettre en place un rafraîchissement incrémental rapide (FAST ON COMMIT) complété par un index d'accès rapide :

```
1 CREATE INDEX idx_mv_dept ON mv_stat_dept(nom_dept);
```

4. **Outils d'analyse** : Analyser régulièrement les coûts d'exécution via les commandes EXPLAIN PLAN afin de corriger les scans complets de tables (*Full Table Scans*).

Conclusion

L'ensemble de ce travail pratique met en évidence l'intérêt fonctionnel des vues SQL dans la conception moderne d'architectures de bases de données. Qu'il s'agisse d'une abstraction logique via une vue simple ou d'une quête de performance décisionnelle au travers de vues matérialisées, leur usage maîtrisé et encadré par des contrôles d'intégrité rigoureux (`WITH CHECK OPTION`) s'avère indispensable pour administrer de façon pérenne des systèmes d'informations d'envergure.

TP 3 : Concepts fondamentaux

Introduction

Ce tutoriel et rapport de travaux pratiques a pour objectif de maîtriser les concepts fondamentaux et avancés du langage procédural d'Oracle, le **PL/SQL**. L'accent est mis sur la déclaration rigoureuse des variables, la délimitation de leur portée, l'utilisation des attributs dynamiques de typage (%TYPE et %ROWTYPE), ainsi que la manipulation des enregistrements (RECORD) et des collections de type tables associatives.

L'ensemble des codes présentés est entièrement valide et exécutable au sein d'environnements de développement tels que *Oracle SQL Developer* ou *SQL*Plus*. Avant toute exécution, l'activation du canal d'affichage est requise via la commande réglementaire :

```
SET SERVEROUTPUT ON;
```

Structure Fondamentale d'un Bloc PL/SQL

Le PL/SQL est un langage structuré par blocs blocs. Un bloc de base obéit à l'architecture suivante :

```
1  DECLARE
2      -- Section optionnelle de declaration des variables et
      types
3  BEGIN
4      -- Section obligatoire contenant les instructions
      executables
5  EXCEPTION
6      -- Section optionnelle de traitement des erreurs (
      exceptions)
7  END ;
8  /
```

17 – Architecture d'un bloc standard

Exercice 1 : Manipulation des Variables Scalaires

Objectifs & Consigne

Manipuler différents types de variables scalaires usuelles (VARCHAR2, NUMBER, DATE) , appliquer des initialisations directes, affecter des expressions de calcul et employer des expressions constantes.

Solution Implémentée

```

1  DECLARE
2      v_nom          VARCHAR2(30) := 'Alice';
3      v_salaire      NUMBER(8,2)  := 3500.50;
4      v_date_emb     DATE          := SYSDATE;
5      c_taux_impot   CONSTANT NUMBER(3,2) := 0.15;
6      v_prime        NUMBER(8,2);
7  BEGIN
8      -- Calcul de la prime sur la base du taux fixe constant
9      v_prime := v_salaire * c_taux_impot;
10
11     -- Affichage du resultat sur la console standard
12     DBMS_OUTPUT.PUT_LINE('Employ : ' || v_nom);
13     DBMS_OUTPUT.PUT_LINE('Prime : ' || v_prime);
14 END;
15 /

```

18 – Déclaration et affectation de variables scalaires

Analyse & Règles Éditées

- **Constantes** : Toute variable déclarée avec le mot-clé **CONSTANT** doit obligatoirement faire l'objet d'une initialisation dès sa ligne de déclaration.
- **Contrainte Not Null** : L'ajout de la clause **NOT NULL** impose formellement une valeur par défaut immédiate lors de l'allocation mémoire.
- **Type Boolean** : Le type **BOOLEAN** est un type purement PL/SQL ; il ne peut être stocké au sein d'une table physique SQL ou être sollicité dans une requête SQL standard.

Exercice 2 : Portée et Visibilité des Variables

Objectifs & Consigne

Mettre en évidence les concepts d'imbrication de blocs, de variables globales et locales, et définir leurs limites de visibilité respectives.

Solution Implémentée

```

1  DECLARE
2      v_externe NUMBER := 10; -- Variable globale au bloc
3  BEGIN
4      DECLARE
5          v_interne NUMBER := 20; -- Variable locale au sous-bloc
6      BEGIN
7          -- Un bloc interne accede a ses variables et aux
              variables externes
8          DBMS_OUTPUT.PUT_LINE('Interne : ' || v_interne);
9          DBMS_OUTPUT.PUT_LINE('Externe vu de l''interne : ' ||
              v_externe);
10     END;
11     -- Tentative d'affichage impossible de v_interne ici :
              provoquerait une erreur
12     DBMS_OUTPUT.PUT_LINE('Externe : ' || v_externe);
13 END;
14 /

```

19 – Illustration des blocs imbriqués

Analyse Conceptuelle

- **Bloc interne** : Possède une visibilité complète sur les variables instanciées dans le bloc parent direct (**v_externe**).
- **Bloc externe** : Est totalement aveugle face aux structures locales isolées au sein des couches inférieures (**v_interne**). La mémoire allouée à **v_interne** est libérée dès la rencontre de la balise **END** du sous-bloc.

Exercice 3 : L'attribut de typage %TYPE

Objectifs & Consigne

Garantir la sécurité du typage applicatif en liant dynamiquement le type d'une variable PL/SQL à la définition d'une colonne physique d'une table existante.

Pour cet exercice, la structure de table de référence est la suivante :

```

1 CREATE TABLE employes (
2     id NUMBER,
3     nom VARCHAR2(50),
4     salaire NUMBER(8,2)
5 );

```

20 – Table physique employes

Solution Implémentée

```

1 DECLARE
2     v_nom      employes.nom%TYPE;
3     v_salaire  employes.salaire%TYPE;
4 BEGIN
5     v_nom      := 'Jean';
6     v_salaire := 5000;
7
8     DBMS_OUTPUT.PUT_LINE(v_nom || ' gagne ' || v_salaire);
9 END;
10 /

```

21 – Utilisation de l'attribut %TYPE

Avantages d'Utilisation

L'ancrage par %TYPE confère trois atouts fondamentaux pour l'ingénierie logicielle :

- **Adaptation structurelle automatique** : Si la précision ou le type de la colonne `salaire` évolue en base (ex : `NUMBER(10,2)`), le code PL/SQL s'adapte sans nécessiter de recompilation ou réécriture manuelle.
- **Maintenance logicielle simplifiée** : Réduction drastique des risques d'incompatibilité entre le stockage et le code applicatif.
- **Sécurisation du typage** : Suppression des erreurs de type *Data Overflow* ou de conversion de chaînes tronquées.

Exercice 4 : L'attribut de ligne %ROWTYPE

Objectifs & Consigne

Déclarer une entité capable de stocker en une seule variable l'intégralité d'un enregistrement physique, calqué sur le modèle complet des colonnes d'une table.

Solution Implémentée

```

1 DECLARE
2     v_emp employees%ROWTYPE; -- Variable contenant id, nom, et
   salaire
3 BEGIN
4     -- Affectation par champs via le selecteur point "."
5     v_emp.id      := 1;
6     v_emp.nom     := 'Maria';
7     v_emp.salaire := 6000;
8
9     DBMS_OUTPUT.PUT_LINE(v_emp.nom || ' a pour salaire ' ||
   v_emp.salaire);
10 END;
11 /

```

22 – Structure d'enregistrement avec %ROWTYPE

Intérêt Technique

- **Compatibilité Ascendante** : Cette structure est idéalement conçue pour réceptionner directement les tuples issus d'extractions de type `SELECT * INTO v_emp FROM employes`.
- **Souplesse Évolutive** : Si de nouvelles colonnes (ex : `date_naissance`) sont ajoutées à la table, l'objet `%ROWTYPE` intègre immédiatement ces propriétés sans altérer la validité du bloc.

Exercice 5 : Les RECORD Personnalisés

Objectifs & Consigne

Concevoir une structure de données composite sur mesure (**RECORD**), dissociée d'une table physique, pour agglomérer logiquement des variables hétérogènes.

Solution Implémentée

```

1  DECLARE
2      -- 1. Definition de la structure du type composite
3      TYPE type_emp IS RECORD (
4          id          NUMBER,
5          nom          VARCHAR2(50),
6          salaire     NUMBER(8,2)
7      );
8
9      -- 2. Instanciation de la variable basee sur le type cree
10     v_emp type_emp;
11 BEGIN
12     v_emp.id          := 2;
13     v_emp.nom          := 'David';
14     v_emp.salaire     := 4500;
15
16     DBMS_OUTPUT.PUT_LINE('Employ enregistrr : ' || v_emp.nom
17                             );
18 END;
19 /

```

23 – Déclaration et usage d'un RECORD personnalisé

Cas d'Usage Préférentiels

L'implémentation d'un **RECORD** personnalisé s'avère incontournable lorsque :

- Le développeur souhaite manipuler un sous-ensemble ou un regroupement hybride de colonnes issues de multiples tables distinctes en jointure.
- Un besoin de factorisation et de regroupement logique de données métiers se manifeste au sein des flux applicatifs complexes.

Exercice 6 : Les Tables PL/SQL (Collections Associatives)

Objectifs & Consigne

Manipuler les concepts de collections dynamiques en créant une table associative indexée par des entiers pour stocker des tableaux de valeurs.

Solution Implémentée

```

1  DECLARE
2      -- D cl aration d'un type table index par entier
3      TYPE table_salaires IS TABLE OF NUMBER INDEX BY
4          BINARY_INTEGER;
5
6      -- Instanciation du tableau
7      v_salaires table_salaires;
8
9  BEGIN
10     -- Remplissage manuel par affectation d'index
11     v_salaires(1) := 3000;
12     v_salaires(2) := 4500;
13     v_salaires(3) := 5000;
14
15     -- Parcours it ratif de la collection
16     FOR i IN 1..3 LOOP
17         DBMS_OUTPUT.PUT_LINE('Salaire      l''index ' || i || ' :
18             ' || v_salaires(i));
19     END LOOP;
20 END;
21 /

```

24 – Utilisation d'une table associative

Caractéristiques Majeures

Les tables associatives se démarquent par trois propriétés fondamentales :

- **Taille dynamique** : Aucun dimensionnement initial rigide n'est imposé ; la mémoire s'étend à l'insertion de nouvelles clés.
- **Structure creuse (Sparse)** : Les indices ne sont pas obligatoirement contigus (il est possible d'avoir un élément à l'index 1 et le suivant à l'index 500 sans consommer de mémoire intermédiaire).
- **Indexation native** : Indexation performante assurée par des entiers signés de type

BINARY_INTEGER.

Exercice 7 : Exercice Intégré Avancé (Combinaisons)

Consigne

Réaliser une imbrication de structures complexes en concevant une collection dynamique (TABLE) dont les cellules contiennent des lignes entières de structures physiques de tables issues du dictionnaire de données (%ROWTYPE).

Solution Implémentée

```

1  DECLARE
2      -- D c l a r a t i o n d ' u n t y p e t a b l e d o n t c h a q u e c e l l u l e e s t u n
        e n r e g i s t r e m e n t c o m p l e t
3      TYPE table_emp IS TABLE OF employes%ROWTYPE INDEX BY
        BINARY_INTEGER;
4
5      v_emps table_emp;
6  BEGIN
7      -- 1. Phase d'alimentation dynamique des cellules via
        b o u c l e F O R
8      FOR i IN 1..3 LOOP
9          v_emps(i).id      := i;
10         v_emps(i).nom      := 'Employ ' || i;
11         v_emps(i).salaire := 3000 + (i * 1000);
12     END LOOP;
13
14     -- 2. Phase de restitution et lecture des donn es
        c o m p o s i t e s
15     FOR i IN 1..3 LOOP
16         DBMS_OUTPUT.PUT_LINE(v_emps(i).nom || ' gagne ' ||
            v_emps(i).salaire);
17     END LOOP;
18 END;
19 /

```

25 – Collection de %ROWTYPE et structures imbriquées

Exercice 8 : Travail Final de Synthèse

Travail demandé

Mettre en pratique l'ensemble des notions du TP en rédigeant un bloc autonome accomplissant les jalons suivants :

1. Définir un RECORD personnalisé décrivant un profil d'employé.
2. Déclarer une TABLE associative stockant ces enregistrements.
3. Effectuer l'insertion de 5 entités distinctes au sein de la collection.
4. Parcourir la structure pour calculer le salaire moyen global.
5. Identifier et afficher le salaire maximum détecté.

Solution de Synthèse Globale

```

1  DECLARE
2      -- 1. Definition du RECORD
3      TYPE rec_stat_emp IS RECORD (
4          emp_id  NUMBER,
5          nom      VARCHAR2(50),
6          salaire  NUMBER(8,2)
7      );
8
9      -- 2. Definition de la TABLE de ce RECORD
10     TYPE tbl_stat_emp IS TABLE OF rec_stat_emp INDEX BY
11         BINARY_INTEGER;
12     v_liste tbl_stat_emp;
13
14     -- Variables d'analyse statistique
15     v_somme_salaires  NUMBER(12,2) := 0;
16     v_salaire_moyen   NUMBER(8,2)  := 0;
17     v_salaire_max     NUMBER(8,2)  := 0;
18     v_total_elements  CONSTANT NUMBER := 5;
19 BEGIN
20     -- 3. Insertion nominale de 5 employes tests
21     v_liste(1).emp_id := 101; v_liste(1).nom := 'Ahmed';
22     v_liste(1).salaire := 4000;
23     v_liste(2).emp_id := 102; v_liste(2).nom := 'Sanaa';
24     v_liste(2).salaire := 7500;
25     v_liste(3).emp_id := 103; v_liste(3).nom := 'Karim';
26     v_liste(3).salaire := 5200;
27     v_liste(4).emp_id := 104; v_liste(4).nom := 'Yassine';
28     v_liste(4).salaire := 8900;

```

```

24     v_liste(5).emp_id := 105; v_liste(5).nom := 'Fatima';
      v_liste(5).salaire := 6100;
25
26     -- 4 & 5. Algorithme de parcours unique pour calculs
      cumulatifs
27     FOR i IN 1..v_total_elements LOOP
28         v_somme_salaires := v_somme_salaires + v_liste(i).
            salaire;
29
30         -- Recherche du maximum absolu
31         IF v_liste(i).salaire > v_salaire_max THEN
32             v_salaire_max := v_liste(i).salaire;
33         END IF;
34     END LOOP;
35
36     v_salaire_moyen := v_somme_salaires / v_total_elements;
37
38     -- Restitution finale des indicateurs agregés
39     DBMS_OUTPUT.PUT_LINE('--- Resultats de la Synthese ---');
40     DBMS_OUTPUT.PUT_LINE('Salaire Moyen Global : ' ||
        v_salaire_moyen || ' DH');
41     DBMS_OUTPUT.PUT_LINE('Salaire Maximum Detecte : ' ||
        v_salaire_max || ' DH');
42 END;
43 /

```

26 – Résolution de l'exercice final de synthèse

Règles de l'Art, Bonnes Pratiques et Pièges Fréquents

Normes et Standards de Codage Préconisés

Pour assurer la maintenabilité des architectures logicielles sous Oracle, les directives suivantes doivent être appliquées par le développeur :

- **Typage Dynamique Systématique** : Privilégier l'usage de %TYPE et %ROWTYPE plutôt que de forcer des types physiques rigides en dur.
- **Principe d'Unicité** : Déclarer impérativement **une seule variable par ligne** de code pour maximiser la lisibilité des blocs.
- **Norme de Nommage par Préfixes** : Standardiser les identifiants selon la nomenclature technique suivante :

TABLE 3 – Règles de préfixage des variables

Préfixe	Destination d'usage	Exemple concret
v_	Variables standards modifiables	v_salaire_net
c_	Constantes en lecture seule	c_taux_tva
t_	Définitions de types utilisateur	t_liste_employes

Pièges Classiques et Erreurs Fréquentes à Proscrire

La vigilance de l'administrateur ou du développeur doit se porter sur les défaillances de logique courantes suivantes :

- **Défaut d'initialisation** : Omettre d'assigner une valeur de départ à des variables soumises à la clause NOT NULL ou à des constantes.
- **Violation d'indice de collection** : Tenter d'accéder en lecture à une clé ou un index de table associative qui n'a pas été préalablement instancié ou initialisé.
- **Confusion de langages** : Vouloir exécuter des variables de type BOOLEAN ou appeler des fonctions d'affichage propres au DBMS_OUTPUT à l'intérieur de requêtes SQL pures.

Conclusion

La maîtrise absolue de la manipulation des variables, des structures composites de types RECORD et des collections associatives constitue le socle technique indispensable à tout développeur s'orientant vers l'ingénierie des bases de données avancées. Ces fondamentaux ouvrent la voie à l'écriture de composants serveurs hautement performants, à l'instar des **procédures stockées**, des **fonctions complexes** et de la modularité offerte par les **packages Oracle**. Une déclaration rigoureuse garantit ainsi un code robuste, pérenne et hautement maintenable.

TP 4 : Interaction avec les BD et Structures de Contrôle

Introduction

Ce rapport présente les concepts fondamentaux du langage procédural d'Oracle (**PL/SQL**) liés à l'interaction directe avec le dictionnaire de données et à la mise en œuvre des structures de contrôle algorithmiques. L'intégration de requêtes SQL (DML) au sein de structures conditionnelles (**IF-THEN-ELSE**) et répétitives (**LOOP**, **WHILE**, **FOR**) permet de concevoir des traitements batchs complexes et performants côté serveur.

Avant toute phase de test, l'ouverture du flux d'affichage standard est requise :

```
SET SERVEROUTPUT ON;
```

Exercices Pratiques

Exercice 1 : Insertion séquentielle via une boucle

Consigne

Créer une table nommée **Nombre** contenant une seule colonne **n** **NUMBER**. Écrire un bloc PL/SQL qui insère dans cette table les nombres de 1 à 100 en utilisant une structure itérative.

Solution Implémentée

```
1  -- 1. Creation de la table physique
2  CREATE TABLE Nombre (
3      n NUMBER
4  );
5
6  -- 2. Bloc PL/SQL d'insertion
7  BEGIN
8      FOR i IN 1..100 LOOP
9          INSERT INTO Nombre (n) VALUES (i);
10         END LOOP;
11         COMMIT; -- Validation de la transaction
12     END;
13 /
```

27 – Création de table et insertion par boucle FOR

Exercice 2 : Calcul cumulatif de somme

Consigne

Écrire un programme PL/SQL qui calcule la somme de tous les nombres compris entre 1 et 1000 et affiche le résultat à l'écran.

Solution Implémentée

```
1  DECLARE
2      v_somme NUMBER := 0;
3  BEGIN
4      FOR i IN 1..1000 LOOP
5          v_somme := v_somme + i;
6      END LOOP;
```

```
7      DBMS_OUTPUT.PUT_LINE('La somme des nombres de 1 a 1000 est  
      : ' || v_somme);  
8  END;  
9  /
```

28 – Calcul cumulatif de 1 a 1000

Exercice 3 : Filtrage et affichage de nombres pairs

Consigne

Écrire un programme PL/SQL qui affiche les nombres pairs compris entre 1 et 50.

Solution Implémentée

```
1  BEGIN  
2      FOR i IN 1..50 LOOP  
3          IF MOD(i, 2) = 0 THEN  
4              DBMS_OUTPUT.PUT_LINE(i || ' est pair');  
5          END IF;  
6      END LOOP;  
7  END;  
8  /
```

29 – Détermination des nombres pairs avec l'opérateur MOD

Exercice 4 : Calcul de la fonction factorielle

Consigne

Écrire un programme PL/SQL qui calcule le factoriel d'un nombre entier n .

Solution Implémentée

```

1 DECLARE
2     v_n          NUMBER := 6; -- Valeur cible pour le calcul
   (6!)
3     v_factoriel  NUMBER := 1;
4 BEGIN
5     IF v_n < 0 THEN
6         DBMS_OUTPUT.PUT_LINE('Le factoriel n''est pas defini
   pour les negatifs.');
```

```

7     ELSIF v_n = 0 THEN
8         DBMS_OUTPUT.PUT_LINE('0! = 1');
```

```

9     ELSE
10        FOR i IN 1..v_n LOOP
11            v_factoriel := v_factoriel * i;
12        END LOOP;
13        DBMS_OUTPUT.PUT_LINE(v_n || '!' = ' || v_factoriel);
14    END IF;
15 END;
16 /
```

30 – Calcul itératif du factoriel d'un entier n

Exercice 5 : Gestion de table et distribution de notes

Consigne

Créer une table Etudiant(id, nom, note). Écrire un bloc PL/SQL qui insère 10 étudiants dans la table en attribuant une note différente à chaque étudiant.

Solution Implémentée

```

1 -- 1. Creation de la table
2 CREATE TABLE Etudiant (
3     id    NUMBER PRIMARY KEY,
4     nom   VARCHAR2(50),
5     note  NUMBER(4,2)
6 );
7
```

```
8  -- 2. Remplissage par bloc PL/SQL
9  BEGIN
10     FOR i IN 1..10 LOOP
11         INSERT INTO Etudiant (id, nom, note)
12         VALUES (i, 'Etudiant_' || i, 10 + (i * 0.8));
13     END LOOP;
14     COMMIT;
15 END;
16 /
```

31 – Génération automatisée d'enregistrements d'étudiants

Exercice 6 : Décompte inverse via une boucle WHILE

Consigne

Écrire un programme PL/SQL qui affiche les nombres de 10 à 1 en utilisant une boucle WHILE.

Solution Implémentée

```

1 DECLARE
2     v_compteur NUMBER := 10;
3 BEGIN
4     WHILE v_compteur >= 1 LOOP
5         DBMS_OUTPUT.PUT_LINE('Valeur : ' || v_compteur);
6         v_compteur := v_compteur - 1; -- Decrementation
           explicite
7     END LOOP;
8 END;
9 /

```

32 – Utilisation de la structure itérative WHILE

Exercice 7 : Mise à jour de masse (Bulk Update)

Consigne

Créer une table `Produit (id, nom, prix)`. Écrire un programme PL/SQL qui augmente le prix de tous les produits de 10%.

Solution Implémentée

```

1 -- 1. Creation de la table Produit
2 CREATE TABLE Produit (
3     id NUMBER PRIMARY KEY,
4     nom VARCHAR2(50),
5     prix NUMBER(10,2)
6 );
7
8 -- 2. Bloc PL/SQL d'augmentation de prix
9 BEGIN
10    UPDATE Produit
11    SET prix = prix * 1.10;
12
13    DBMS_OUTPUT.PUT_LINE('Mise a jour reussie. ' || SQL%
        ROWCOUNT || ' produits modifies.');
```

```
14      COMMIT ;  
15  END ;  
16  /
```

33 – Traitement de mise à jour globale

Exercice 8 : Suppression conditionnelle

Consigne

Écrire un programme PL/SQL qui supprime tous les produits dont le prix est strictement inférieur à 50.

Solution Implémentée

```

1 BEGIN
2     DELETE FROM Produit
3     WHERE prix < 50;
4
5     DBMS_OUTPUT.PUT_LINE('Suppression terminee. Nbr de lignes
6         impactees : ' || SQL%ROWCOUNT);
7     COMMIT;
8 END;
9 /

```

34 – Opération DML de suppression avec suivi SQL%ROWCOUNT

Exercice 9 : Recherche et traitement conditionnel d'un produit

Consigne

Écrire un programme PL/SQL qui cherche le prix du produit ayant l'id = 1. Si le prix est supérieur à 100, afficher « Cher », sinon afficher « Abordable ». Si le produit n'existe pas, gérer l'exception de manière appropriée.

Solution Implémentée

```

1 DECLARE
2     v_prix Produit.prix%TYPE;
3 BEGIN
4     -- Extraction securisee du prix pour l'ID specifie
5     SELECT prix INTO v_prix
6     FROM Produit
7     WHERE id = 1;
8
9     -- Structure conditionnelle d'analyse
10    IF v_prix > 100 THEN
11        DBMS_OUTPUT.PUT_LINE('Le produit d''ID 1 est : Cher');
12    ELSE

```

```
13         DBMS_OUTPUT.PUT_LINE('Le produit d''ID 1 est :  
           Abordable');  
14     END IF;  
15  
16 EXCEPTION  
17     WHEN NO_DATA_FOUND THEN  
18         DBMS_OUTPUT.PUT_LINE('Erreur : Aucun produit ne possede  
           1''ID 1.');
```

```
19     WHEN TOO_MANY_ROWS THEN  
20         DBMS_OUTPUT.PUT_LINE('Erreur : Coherence corrompue,  
           plusieurs lignes detectees.');
```

```
21 END;  
22 /
```

35 – Extraction SELECT INTO et capture des exceptions

Analyse Méthodologique et Structures Fondamentales

Comparatif des structures itératives

L'ingénierie des blocs PL/SQL impose de choisir la structure de boucle la plus adaptée aux besoins fonctionnels et de performance :

TABLE 4 – Matrice d'évaluation des boucles en PL/SQL

Structure	Condition de test	Nombre d'itérations	Cas d'usage recommandé
LOOP	Interne (EXIT WHEN)	Au moins 1 fois	Sortie sur critère dynamique
WHILE	Initiale (En entrée)	Variable (≥ 0)	Attente d'un changement d'état
FOR	Automatique (Bornes)	Connu et fixe	Parcours d'intervalles / curseurs

Gestion de transactions et curseurs implicites

- **SQL%ROWCOUNT** : Cet attribut de curseur implicite est indispensable après chaque commande UPDATE ou DELETE. Il permet de connaître instantanément le nombre précis de lignes affectées par la dernière instruction SQL.
- **COMMIT / ROLLBACK** : Toute opération d'écriture (DML) initiée dans un bloc PL/SQL doit être finalisée de manière atomique. L'appel à COMMIT valide définitivement les modifications en base, tandis que les sections EXCEPTION doivent idéalement contenir un ROLLBACK pour préserver l'intégrité des données en cas d'anomalie.

Conclusion

Ce travail pratique met en relief la puissance de l'imbrication du langage procédural PL/SQL avec la flexibilité du langage relationnel SQL. L'utilisation rigoureuse des structures de contrôle assure une automatisation complète des opérations de maintenance de données, tandis que la capture systématique des exceptions (telles que NO_DATA_FOUND) garantit la production de scripts serveurs industriels, hautement sécurisés et tolérants aux pannes.

TP 5 : Curseurs PL-SQL

Introduction

Un curseur est une structure ou zone mémoire de travail (appelée bloc de contexte) allouée par le serveur Oracle pour exécuter une requête SQL et stocker les informations associées. Contrairement au **SELECT INTO** qui exige le retour strict d'un unique tuple, le curseur explicite offre l'agilité nécessaire pour traiter individuellement, ligne par ligne, un ensemble volumineux d'enregistrements.

Le cycle de vie d'un curseur explicite comprend quatre étapes fondamentales :

1. **Déclaration (CURSOR ... IS)** : Définition nominative de la requête de sélection dans la section **DECLARE**.
2. **Ouverture (OPEN)** : Analyse syntaxique, allocation mémoire et exécution de la requête pour charger le jeu de résultats.
3. **Extraction (FETCH)** : Récupération de la ligne courante du curseur dans des variables locales et avancée automatique du pointeur.
4. **Fermeture (CLOSE)** : Libération complète de la mémoire allouée au bloc de contexte.

Pour exécuter les blocs de ce rapport et recevoir les flux d'affichage, la configuration initiale requise est :

```
SET SERVEROUTPUT ON;
```


Exercice 1 : Gestion de la Base de Données Usine

Schéma Relationnel de Référence

- **Commande** (Numcmd, Datcmd, #Numfour, Mncmd)
- **Fournisseurs** (Numfour, Adrfour)

Question 1 : Liste complète des commandes

Consigne

En utilisant un curseur sous PL/SQL, donnez la liste complète des commandes présentes dans la base.

Solution Implémentée

```

1  DECLARE
2      -- Declaration du curseur explicite
3      CURSOR c_commandes IS
4          SELECT Numcmd, Datcmd, Mncmd FROM Commande;
5
6      -- Variable de reception calquee sur la structure du
7      -- curseur
8      v_cmd c_commandes%ROWTYPE;
9
10 BEGIN
11     OPEN c_commandes;
12     LOOP
13         FETCH c_commandes INTO v_cmd;
14         EXIT WHEN c_commandes%NOTFOUND; -- Condition d'arret du
15                                         -- parcours
16
17         DBMS_OUTPUT.PUT_LINE('Commande N : ' || v_cmd.Numcmd
18                               ||
19                               ' | Date: ' || TO_CHAR(v_cmd.
20                                                       Datcmd, 'DD/MM/YYYY') ||
21                               ' | Montant: ' || v_cmd.Mncmd);
22     END LOOP;
23     CLOSE c_commandes;
24 END;
25 /

```

36 – Curseur standard pour l'affichage des commandes

Question 2 : Commandes supérieures à la moyenne

Consigne

Donnez la liste des commandes ayant un montant (Mncmd) strictement supérieur à la moyenne globale des montants de toutes les commandes.

Solution Implémentée

```

1 DECLARE
2     -- Insertion de la sous-requete de calcul de moyenne
3     CURSOR c_cmd_sup_moy IS
4         SELECT Numcmd, Mncmd
5         FROM Commande
6         WHERE Mncmd > (SELECT AVG(Mncmd) FROM Commande);
7
8     v_cmd c_cmd_sup_moy%ROWTYPE;
9 BEGIN
10    DBMS_OUTPUT.PUT_LINE('--- Commandes superieures a la
11                           moyenne ---');
12    OPEN c_cmd_sup_moy;
13    LOOP
14        FETCH c_cmd_sup_moy INTO v_cmd;
15        EXIT WHEN c_cmd_sup_moy%NOTFOUND;
16
17        DBMS_OUTPUT.PUT_LINE('ID Commande: ' || v_cmd.Numcmd ||
18                              ' | Montant: ' || v_cmd.Mncmd);
19    END LOOP;
20    CLOSE c_cmd_sup_moy;
21 END;
22 /

```

37 – Utilisation d'une sous-requete aggregate au sein du curseur

Question 3 : Commandes de fournisseurs parisiens

Consigne

Donnez la liste des commandes passées auprès d'un fournisseur basé à Paris.

Solution Implémentée

```

1 DECLARE
2     CURSOR c_cmd_paris IS
3         SELECT c.Numcmd, c.Mncmd, f.Adrfour
4         FROM Commande c
5         JOIN Fournisseurs f ON c.Numfour = f.Numfour
6         WHERE UPPER(f.Adrfour) = 'PARIS';
7
8     v_cmd c_cmd_paris%ROWTYPE;
9 BEGIN
10    OPEN c_cmd_paris;
11    LOOP
12        FETCH c_cmd_paris INTO v_cmd;
13        EXIT WHEN c_cmd_paris%NOTFOUND;
14
15        DBMS_OUTPUT.PUT_LINE('Commande Parisienne N : ' ||
16                               v_cmd.Numcmd || ' | Montant: ' || v_cmd.Mncmd);
17    END LOOP;
18    CLOSE c_cmd_paris;
19 END;
/

```

Exercice 2 : Gestion des Employés de la Société Gava-soft

Schéma Relationnel de Référence

- **Emp** (NumE, NomE, Fonction, NumS, Embauche, Salaire, Comm, #NumD)
- **Dept** (NumD, NomD, Lieu)

Question 1 : Augmentation sélective des salaires (Curseur paramétré)

Consigne

Augmenter le salaire de 5% pour les employés occupant une fonction passée en paramètre, et de 10% pour les autres.

Solution Implémentée

```

1  DECLARE
2      -- Utilisation d'un curseur global sur les employes
3      CURSOR c_employes IS
4          SELECT NumE, Fonction, Salaire FROM Emp FOR UPDATE;
5
6      v_param_fonction Emp.Fonction%TYPE := 'Stagiaire'; --
          Fonction cible
7      v_emp c_employes%ROWTYPE;
8      v_nouv_salaire Emp.Salaire%TYPE;
9  BEGIN
10     OPEN c_employes;
11     LOOP
12         FETCH c_employes INTO v_emp;
13         EXIT WHEN c_employes%NOTFOUND;
14
15         -- Verification de la fonction pour appliquer le taux
            correct
16         IF v_emp.Fonction = v_param_fonction THEN
17             v_nouv_salaire := v_emp.Salaire * 1.05;
18         ELSE
19             v_nouv_salaire := v_emp.Salaire * 1.10;
20         END IF;
21
22         -- Application de la modification ciblée
23         UPDATE Emp

```

```
24         SET Salaire = v_nouv_salaire
25         WHERE CURRENT OF c_employes;
26     END LOOP;
27     CLOSE c_employes;
28     COMMIT; -- Securisation des ecritures
29 END;
30 /
```

39 – Mise a jour de donnees via curseur et structure conditionnelle

Question 2 : Affichage hiérarchisé Employé — Département

Consigne

Afficher la liste des employés avec le nom et le lieu de leur département respectif.

Solution Implémentée (Optimisée via boucle FOR)

```

1  DECLARE
2      -- Jointure complete pour l'extraction croisee
3      CURSOR c_emp_dept IS
4          SELECT e.NomE, e.Fonction, d.NomD, d.Lieu
5          FROM Emp e
6          LEFT JOIN Dept d ON e.NumD = d.NumD;
7  BEGIN
8      -- La boucle FOR automatise OPEN, FETCH, la declaration du
      -- record et CLOSE
9      FOR rc IN c_emp_dept LOOP
10         DBMS_OUTPUT.PUT_LINE('Employ   : ' || rc.NomE ||
11                               ' | Poste : ' || rc.Fonction ||
12                               ' | Dept  : ' || NVL(rc.NomD, '
13                               Aucun') ||
14                               ' | Ville : ' || NVL(rc.Lieu, 'N/A
15                               '));
16     END LOOP;
17 END;
18 /

```

40 – Parcours simplifié avec une boucle FOR de curseur

Question 3 : Traitement d'un département spécifique (Curseur à argument)

Consigne

Donner la liste des employés travaillant dans un département dont le numéro est saisi ou passé en paramètre.

Solution Implémentée

```

1  DECLARE
2      -- Declaration d'un curseur acceptant un parametre en
      -- entree
3      CURSOR c_emp_par_dept (p_num_dept Dept.NumD%TYPE) IS

```

```
4      SELECT NomE, Salaire
5      FROM Emp
6      WHERE NumD = p_num_dept;
7
8      v_dept_cible Dept.NumD%TYPE := 1; -- Exemple : Departement
      de Droit
9  BEGIN
10     DBMS_OUTPUT.PUT_LINE('--- Liste du personnel du departement
      ' || v_dept_cible || ' ---');
11     -- Passage de la variable lors de l'ouverture du curseur
12     FOR rc IN c_emp_par_dept(v_dept_cible) LOOP
13         DBMS_OUTPUT.PUT_LINE('Nom : ' || rc.NomE || ' | Salaire
      : ' || rc.Salaire);
14     END LOOP;
15 END;
16 /
```

41 – Mise en oeuvre d'un curseur avec argument de filtrage

Synthèse et Matrice d'Évaluation des Propriétés

Attributs fondamentaux des curseurs explicites

Pour contrôler l'état d'avancement des extractions et structurer les conditions d'arrêt, PL/SQL fournit quatre attributs majeurs :

TABLE 5 – Propriétés et comportement des attributs de curseur

Attribut	Signification technique
%FOUND	Renvoie TRUE si le dernier FETCH a extrait une ligne valide.
%NOTFOUND	Renvoie TRUE si le curseur est vide ou si l'on dépasse la fin du jeu.
%ISOPEN	Renvoie TRUE si l'objet mémoire est actuellement ouvert et actif.
%ROWCOUNT	Restitue le nombre cumulé de lignes extraites jusqu'à présent.

Optimisation de la modification des données

Lorsqu'un curseur est créé dans le but de modifier ou supprimer les lignes lues (comme illustré à l'Exercice 2, Question 1), deux clauses de programmation avancées doivent être associées :

- **FOR UPDATE** : Placé à la fin de la requête du curseur, il verrouille physiquement les lignes sélectionnées lors de l'ouverture (**OPEN**), empêchant toute autre transaction concurrente de perturber la mise à jour.
- **WHERE CURRENT OF** : Utilisé dans les commandes **UPDATE** ou **DELETE**, il évite de réécrire une clause **WHERE** complexe en ciblant directement et précisément la ligne sur laquelle le pointeur du curseur est positionné.

Conclusion

La mise en œuvre des curseurs explicites représente une étape majeure dans l'écriture de scripts PL/SQL performants. Qu'ils soient structurés de manière classique (avec gestion manuelle de l'ouverture et de la fermeture) ou optimisés à l'aide de la syntaxe condensée des boucles **FOR**, les curseurs offrent aux développeurs et aux administrateurs de bases de données une maîtrise totale des flux de données complexes, tout en garantissant la cohérence et l'intégrité transactionnelle.

TP 6 : Procédures stockées et fonctions en PL/SQL

Rappels de cours & Méthodologie

L'objet de ce travail pratique est l'étude et le déploiement des objets de code stockés côté serveur : les **Procédures** et les **Fonctions**. Ces blocs nommés permettent de centraliser les règles métiers, d'optimiser le trafic réseau en exécutant les traitements au plus près des fichiers de données, et de structurer la sécurité des applications.

Modes de transmission des paramètres

- **IN (par défaut)** : Le paramètre est passé en lecture seule. Sa valeur ne peut être altérée au sein du sous-programme.
- **OUT** : Le paramètre est utilisé pour retourner une valeur au bloc appelant. Il est initialisé à NULL à l'entrée du sous-programme.
- **IN OUT** : Le paramètre agit à la fois comme une variable d'entrée (lecture) et une variable de sortie (écriture/modification).

Pour capturer et afficher les sorties consoles au cours des phases de test, la commande suivante est activée :

```
SET SERVEROUTPUT ON;
```

Mise en œuvre des Procédures et Fonctions Stockées

Exercice 1 : Affichage du nom d'un employé

Consigne

Créer une procédure stockée qui prend en paramètre l'identifiant d'un employé et affiche son nom. Gérer l'erreur si l'employé n'existe pas.

Solution Implémentée

```

1 CREATE OR REPLACE PROCEDURE afficher_nom_employe (
2     p_enum IN employes.enum%TYPE
3 ) IS
4     v_nom employes.nom%TYPE;
5 BEGIN
6     SELECT nom INTO v_nom
7     FROM employes
8     WHERE enum = p_enum;
9
10    DBMS_OUTPUT.PUT_LINE('L''employé d''ID ' || p_enum || ' se
        nomme : ' || v_nom);
11 EXCEPTION
12     WHEN NO_DATA_FOUND THEN
13         DBMS_OUTPUT.PUT_LINE('Erreur : Aucun employé ne
        poss de l''identifiant ' || p_enum);
14 END;
15 /

```

42 – Procédure d'affichage nominal

Exercice 2 : Calcul du salaire annuel

Consigne

Créer une fonction stockée qui prend en paramètre l'identifiant d'un employé et retourne son salaire annuel (salaire mensuel $\times$ 12).

Solution Implémentée

```

1 CREATE OR REPLACE FUNCTION obtenir_salaire_annuel (
2     p_enum IN employes.enum%TYPE
3 ) RETURN NUMBER IS
4     v_sal_mensuel employes.salaire%TYPE;

```

```
5 BEGIN
6     SELECT salaire INTO v_sal_mensuel
7     FROM employes
8     WHERE enum = p_enum;
9
10    RETURN (v_sal_mensuel * 12);
11 EXCEPTION
12     WHEN NO_DATA_FOUND THEN
13         RETURN 0; -- Retour par défaut en cas d'absence du
                    tuple
14 END;
15 /
```

43 – Fonction de calcul de revenus annuels

Exercice 3 : Augmentation globale des salaires par département

Consigne

Créer une procédure stockée permettant d'augmenter les salaires de tous les employés d'un département donné selon un pourcentage spécifié.

Solution Implémentée

```

1 CREATE OR REPLACE PROCEDURE augmenter_salaire_dept (
2     p_num_dept    IN  employes.num_dept%TYPE,
3     p_pourcentage IN  NUMBER
4 ) IS
5 BEGIN
6     UPDATE employes
7     SET salaire = salaire * (1 + (p_pourcentage / 100))
8     WHERE num_dept = p_num_dept;
9
10    DBMS_OUTPUT.PUT_LINE('Mise      jour effectu e. Nombre d''
        employ s augment s : ' || SQL%ROWCOUNT);
11 END;
12 /

```

44 – Procédure de mise à jour groupée

Exercice 4 : Récupération des informations (Mode OUT)

Consigne

Créer une procédure qui retourne le nom et le salaire d'un employé via des paramètres en mode OUT.

Solution Implémentée

```

1 CREATE OR REPLACE PROCEDURE extraire_infos_employe (
2     p_enum        IN  employes.enum%TYPE,
3     p_nom         OUT employes.nom%TYPE,
4     p_salaire     OUT employes.salaire%TYPE
5 ) IS
6 BEGIN
7     SELECT nom, salaire INTO p_nom, p_salaire
8     FROM employes
9     WHERE enum = p_enum;
10 EXCEPTION

```

```

11     WHEN NO_DATA_FOUND THEN
12         p_nom      := 'INCONNU';
13         p_salaire := 0;
14     END;
15 /

```

45 – Extraction de données avec paramètres OUT

Exercice 5 : Inversion de valeurs (Mode IN OUT)

Consigne

Créer une procédure qui permute les valeurs de deux variables transmises en paramètre.

Solution Implémentée

```

1  CREATE OR REPLACE PROCEDURE permuter_valeurs (
2      p_val1 IN OUT NUMBER,
3      p_val2 IN OUT NUMBER
4  ) IS
5      v_temp NUMBER;
6  BEGIN
7      v_temp := p_val1;
8      p_val1 := p_val2;
9      p_val2 := v_temp;
10 END;
11 /

```

46 – Algorithme de permutation à l'aide de variables temporaires

Exercice 6 : Embauche d'un nouvel employé

Consigne

Créer une procédure d'insertion d'un nouvel employé, avec vérification préalable du montant du salaire (qui doit impérativement être supérieur à zéro).

Solution Implémentée

```

1  CREATE OR REPLACE PROCEDURE inserer_employe (
2      p_enum      IN employes.enum%TYPE,
3      p_nom       IN employes.nom%TYPE,
4      p_salaire   IN employes.salaire%TYPE,
5      p_num_dept  IN employes.num_dept%TYPE
6  ) IS

```

```

7      ex_salaire_invalide EXCEPTION;
8  BEGIN
9      -- Regle metier : verification de validite du salaire
10     IF p_salaire <= 0 THEN
11         RAISE ex_salaire_invalide;
12     END IF;
13
14     INSERT INTO employes (enum, nom, salaire, num_dept)
15     VALUES (p_enum, p_nom, p_salaire, p_num_dept);
16     COMMIT;
17
18 EXCEPTION
19     WHEN ex_salaire_invalide THEN
20         RAISE_APPLICATION_ERROR(-20001, 'Erreur de saisie : Le
           salaire doit tre strictement positif. ');
21     WHEN DUP_VAL_ON_INDEX THEN
22         RAISE_APPLICATION_ERROR(-20002, 'Erreur d''int grit
           : Cet identifiant d''employ existe d j . ');
23 END;
24 /

```

47 – Insertion contrôlée et levée d'exception personnalisée

Exercice 7 : Classement des salaires

Consigne

Créer une fonction stockée qui analyse le salaire d'un employé et retourne un libellé textuel qualifiant son niveau de rémunération (« Élevé », « Moyen », « Faible »).

Solution Implémentée

```

1  CREATE OR REPLACE FUNCTION evaluer_classe_salaire (
2      p_enum IN employes.enum%TYPE
3  ) RETURN VARCHAR2 IS
4      v_salaire employes.salaire%TYPE;
5  BEGIN
6      SELECT salaire INTO v_salaire
7      FROM employes
8      WHERE enum = p_enum;
9
10     -- Application de la logique d' valuation des tranches
11     IF v_salaire > 8000 THEN
12         RETURN ' lev ';

```

```

13     ELSIF v_salaire BETWEEN 4000 AND 8000 THEN
14         RETURN 'Moyen';
15     ELSE
16         RETURN 'Faible';
17     END IF;
18 EXCEPTION
19     WHEN NO_DATA_FOUND THEN
20         RETURN 'Profil Inexistant';
21 END;
22 /

```

48 – Structure conditionnelle d'évaluation

Exercice 8 : Modification de l'adresse d'un employé

Consigne

Créer une procédure permettant de mettre à jour l'adresse d'un employé, identifié par son nom. Gérer les cas d'erreur et intégrer des validations de chaîne non vide.

Solution Implémentée

```

1 CREATE OR REPLACE PROCEDURE modifier_adresse_par_nom (
2     p_nom          IN employes.nom%TYPE,
3     p_adresse      IN VARCHAR2
4 ) IS
5     ex_adresse_vide EXCEPTION;
6 BEGIN
7     -- Validation logicielle : verification de non-vacuit
8     IF p_adresse IS NULL OR TRIM(p_adresse) = '' THEN
9         RAISE ex_adresse_vide;
10    END IF;
11
12    UPDATE employes
13    SET adresse = p_adresse
14    WHERE UPPER(nom) = UPPER(p_nom);
15
16    IF SQL%ROWCOUNT = 0 THEN
17        RAISE NO_DATA_FOUND;
18    END IF;
19    COMMIT;
20
21 EXCEPTION
22    WHEN ex_adresse_vide THEN

```

```

23      RAISE_APPLICATION_ERROR(-20003, 'L''adresse transmise
        ne peut pas tre vide. ');
24  WHEN NO_DATA_FOUND THEN
25      RAISE_APPLICATION_ERROR(-20004, 'Aucun employ trouv
        au nom de : ' || p_nom);
26  WHEN TOO_MANY_ROWS THEN
27      RAISE_APPLICATION_ERROR(-20005, 'Erreur de ciblage :
        Plusieurs employ s poss dent ce nom. ');
28  END;
29  /

```

49 – Procédure de mise à jour d'adresse

Réponses aux questions supplémentaires de l'Exercice 8

- 1. Problématique des homonymes (Nom non unique) :** Si plusieurs employés portent le même nom, l'instruction UPDATE modifiera sans distinction l'adresse de l'ensemble de ces personnes. C'est un comportement dangereux qui brise la cohérence des données. Pour éviter cela, il convient de lever l'exception TOO_MANY_ROWS ou de forcer l'usage de la clé primaire (enum).
- 2. Version alternative de la procédure utilisant le matricule (ENUM) :**

```

1  CREATE OR REPLACE PROCEDURE modifier_adresse_par_enum (
2      p_enum      IN employes.enum%TYPE,
3      p_adresse   IN VARCHAR2
4  ) IS
5  BEGIN
6      IF p_adresse IS NULL OR TRIM(p_adresse) = '' THEN
7          RAISE_APPLICATION_ERROR(-20003, 'L''adresse ne
            peut tre vide. ');
8      END IF;
9
10     UPDATE employes SET adresse = p_adresse WHERE enum =
        p_enum;
11
12     IF SQL%ROWCOUNT = 0 THEN
13         RAISE_APPLICATION_ERROR(-20004, 'Identifiant
            introuvable. ');
14     END IF;
15     COMMIT;
16 END;

```


Exercice 9 : Analyse statistique d'un département

Consigne

Créer une fonction stockée qui retourne le nombre total d'employés rattachés à un département donné.

Solution Implémentée

```

1 CREATE OR REPLACE FUNCTION compter_employes_dept (
2     p_num_dept IN employes.num_dept%TYPE
3 ) RETURN NUMBER IS
4     v_total_personnel NUMBER := 0;
5 BEGIN
6     SELECT COUNT(*) INTO v_total_personnel
7     FROM employes
8     WHERE num_dept = p_num_dept;
9
10    RETURN v_total_personnel;
11 END;
12 /

```

50 – Fonction d'agrégation statistique pour un département

Réponses aux questions supplémentaires de l'Exercice 9

1. Différence fondamentale entre COUNT(*) et COUNT(ENUM) :

- COUNT(*) comptabilise l'intégralité des lignes retournées par la sélection, y compris celles qui contiennent d'éventuelles valeurs NULL.
- COUNT(ENUM) comptabilise uniquement les lignes où le champ ENUM possède une valeur non nulle. (Dans le cas d'une clé primaire, les deux résultats sont rigoureusement identiques).

2. Version modifiée retournant également le salaire moyen (via un type Enregistrement) :

```

1 CREATE OR REPLACE PROCEDURE obtenir_stats_dept (
2     p_num_dept IN employes.num_dept%TYPE,
3     p_total OUT NUMBER,
4     p_moyenne OUT NUMBER
5 ) IS
6 BEGIN
7     SELECT COUNT(*), NVL(AVG(salaire), 0)
8     INTO p_total, p_moyenne

```

```
9      FROM employees
10     WHERE num_dept = p_num_dept;
11 END;
12 /
```

3. Procédure globale de restitution des effectifs par département :

```
1 CREATE OR REPLACE PROCEDURE afficher_repartition_globale
  IS
2 BEGIN
3     FOR rc IN (SELECT num_dept, COUNT(*) as effectif
4                 FROM employees GROUP BY num_dept) LOOP
5         DBMS_OUTPUT.PUT_LINE('D partement N : ' || rc.
6                               num_dept ||
7                               ' | Nombre d''employ s : '
8                               || rc.effectif);
9     END LOOP;
10 END;
11 /
```

Exercice 10 : Purge conditionnelle (Suppression des bas salaires)

Consigne

Créer une procédure stockée qui supprime tous les employés dont le salaire est strictement inférieur à un montant de référence transmis en paramètre.

Solution Implémentée (Avec clause RETURNING et SAVEPOINT)

```

1 CREATE OR REPLACE PROCEDURE purger_bas_salaires (
2     p_salaire_seuil IN employes.salaire%TYPE
3 ) IS
4 BEGIN
5     -- Etablissement d'un point de sauvegarde transactionnel
6     SAVEPOINT sp_avant_purge;
7
8     DELETE FROM employes
9     WHERE salaire < p_salaire_seuil;
10
11     DBMS_OUTPUT.PUT_LINE('Purge termin e. Nombre de profils
12                          supprim s : ' || SQL%ROWCOUNT);
13
14     -- Nous laissons la gestion du COMMIT au bloc applicatif
15     parent externe
16 EXCEPTION
17     WHEN OTHERS THEN
18         ROLLBACK TO sp_avant_purge; -- Restauration de l' tat
19         initial
20         RAISE_APPLICATION_ERROR(-20099, 'Une erreur syst me a
21         provoqu l''annulation de la purge.');
```

51 – Procédure avancée de suppression transactionnelle

Réponses aux questions supplémentaires de l'Exercice 10

1. **Danger de l'utilisation de COMMIT au sein d'une procédure appelée par un programme externe :** L'incorporation de la commande COMMIT dans un sous-programme brise le principe d'atomicité et d'isolation des transactions. Si le programme externe réalise une suite d'opérations connexes (ex : débiter un compte puis créditer un autre), l'exécution d'un COMMIT intermédiaire forcé au milieu du traitement empêchera tout retour en arrière (ROLLBACK) global en cas d'erreur ultérieure dans la chaîne applicative.

2. **Ajout d'une clause RETURNING pour lister les noms supprimés :** Pour récupérer plusieurs noms lors d'une suppression de masse, il est nécessaire d'utiliser la syntaxe RETURNING ... BULK COLLECT INTO associée à une collection de chaînes :

```

1  -- Syntaxe pour capture de masse ( intégrer dans le
    bloc)
2  TYPE t_noms IS TABLE OF employes.nom%TYPE;
3  v_noms_supprimes t_noms;
4  -- SQL: DELETE FROM employes WHERE... RETURNING nom BULK
    COLLECT INTO v_noms_supprimes;

```

Synthèse comparative : Procédure vs Fonction

Afin d'ancrer de manière définitive la gouvernance et le choix d'implémentation de ces deux objets d'architecture de données, voici la matrice de synthèse technique d'Oracle :

TABLE 6 – Matrice d'arbitrage de l'ingénierie PL/SQL

Critère d'évaluation	Procédure	Fonction
Retourne une valeur	Non (mais possède 0 à N paramètres OUT)	Oui, obligatoirement une seule valeur via RETURN
Appel dans une requête SQL	Impossible (Ne s'exécute pas dans un SELECT)	Totalement valide (si elle ne modifie pas la base)
Transaction possible	Oui (COMMIT, ROLLBACK, SAVEPOINT)	Fortement déconseillé (interdit si appelée par un SELECT)
Utilisation typique	Réalisation d'actions métiers, purges, batchs DML	Calculs, formatage, transformations de données

Conclusion

Ce travail de programmation avancée met en lumière l'importance d'isoler la logique applicative au cœur du serveur de base de données. L'utilisation conjointe des procédures pour orchestrer les traitements et des fonctions pour encapsuler les règles de calcul offre un cadre hautement maintenable. De plus, la gestion rigoureuse des exceptions permet de transformer de simples requêtes d'écriture en un système robuste, capable de résister aux erreurs de saisie et de préserver l'intégrité globale des données.

TP 7 : Les curseurs

Exercices : Logique Aléatoire et Instruction IF-THEN-ELSE

Exercice 1 : Réorganisation de variables (Tri croissant)

Consigne

Écrire un programme PL/SQL permettant de réorganiser les valeurs de deux variables de telle sorte que la plus petite valeur soit stockée dans la variable `small_nbr` et la plus grande dans `big_nbr`.

Solution Implémentée

```

1  DECLARE
2      small_nbr  NUMBER := 25;
3      big_nbr    NUMBER := 5;
4      v_temp     NUMBER;
5  BEGIN
6      DBMS_OUTPUT.PUT_LINE('Avant traitement -> small_nbr = ' ||
7                             small_nbr || ', big_nbr = ' || big_nbr);
8
9      -- Verification et permutation si necessaire
10     IF small_nbr > big_nbr THEN
11         v_temp     := small_nbr;
12         small_nbr := big_nbr;
13         big_nbr    := v_temp;
14     END IF;
15
16     DBMS_OUTPUT.PUT_LINE('Sortie -> small_nbr = ' || small_nbr
17                             || '; big_nbr = ' || big_nbr || ');');
18 END;
19 /

```

52 – Algorithme de permutation conditionnelle

Exercice 2 : Vérification de la parité d'un nombre

Consigne

Écrire un programme PL/SQL pour vérifier si un nombre est pair ou impair.

Solution Implémentée

```
1 DECLARE
2     v_nbr NUMBER := 3;
3 BEGIN
4     IF MOD(v_nbr, 2) = 0 THEN
5         DBMS_OUTPUT.PUT_LINE('Le nombre ' || v_nbr || ' est un
6             nombre pair.');
```

```
6     ELSE
7         DBMS_OUTPUT.PUT_LINE('Le nombre ' || v_nbr || ' est un
8             nombre impair.');
```

```
8     END IF;
9 END;
10 /
```

53 – Contrôle de parité via l'opérateur MOD

Exercice 3 : Détection des jours de week-end

Consigne

Écrire un programme PL/SQL permettant de vérifier si une date tombe un jour de week-end (Samedi ou Dimanche).

Solution Implémentée

```

1 DECLARE
2     v_date1 DATE := TO_DATE('02-03-2024', 'DD-MM-YYYY'); -- 2
           Mars 2024
3     v_jour VARCHAR2(20);
4 BEGIN
5     -- Extraction du nom du jour en anglais pour correspondre à
           l'enonce
6     v_jour := TRIM(TO_CHAR(v_date1, 'DAY', 'NLS_DATE_LANGUAGE =
           ENGLISH'));
7
8     IF v_jour = 'SATURDAY' OR v_jour = 'SUNDAY' THEN
9         DBMS_OUTPUT.PUT_LINE('Le jour de la date donnee est '
           || v_jour || ' : tombe le week-end.');
```

```

10    ELSE
11        DBMS_OUTPUT.PUT_LINE('Le jour de la date donnee est '
           || v_jour || ' : jour de semaine de travail.');
```

```

12    END IF;
13 END;
14 /
```

54 – Analyse de date dépendante de la langue de session

Exercices : Manipulation DML et Suivi Transact

Exercice 4 : Copie en masse d'enregistrements (Bulk Copy)

Consigne

Écrire un programme en PL/SQL pour copier les enregistrements de la table `clients` vers une table temporaire nommée `tmp_clients`.

Solution Implémentée

```

1 -- Prerequis : Structure de la table tmp_clients calquee sur
           clients
```

```
2  -- CREATE TABLE tmp_clients AS SELECT * FROM clients WHERE 1=0;
3
4  BEGIN
5      -- Nettoyage preventif de la table temporaire
6      EXECUTE IMMEDIATE 'TRUNCATE TABLE tmp_clients';
7
8      -- Insertion par copie en bloc
9      INSERT INTO tmp_clients
10     SELECT client_id, nom, ville, age FROM clients;
11
12     DBMS_OUTPUT.PUT_LINE('Copie terminee. ' || SQL%ROWCOUNT ||
13         ' lignes transferees avec succes.');
```

55 – Duplication de table et vidage DML

Exercice 5 : Suivi d'impact de mise à jour avec SQL%NOTFOUND

Consigne

Écrire un programme en PL/SQL pour montrer l'utilisation de l'attribut SQL%NOTFOUND afin de déterminer si une instruction UPDATE a effectivement modifié des lignes dans la table pays.

Solution Implémentée

```

1  DECLARE
2      v_id_cible pays.pays_id%TYPE := 1004; -- Identifiant a
      tester
3  BEGIN
4      -- Tentative de mise a jour
5      UPDATE pays
6      SET nom = UPPER(nom)
7      WHERE pays_id = v_id_cible;
8
9      -- Evaluation du curseur implicite SQL
10     IF SQL%NOTFOUND THEN
11         DBMS_OUTPUT.PUT_LINE('Echec de la mise a jour : Aucun
              pays ne possede l''id ' || v_id_cible);
12     ELSE
13         DBMS_OUTPUT.PUT_LINE('Mise a jour reussie. Lignes
              affectees : ' || SQL%ROWCOUNT);
14         COMMIT;
15     END IF;
16 END;
17 /

```

56 – Utilisation des attributs de curseur implicite

Exercice Avancé : Traitement ciblé via WHERE CURRENT OF

Consigne

Créer un bloc PL/SQL pour augmenter le salaire des employés du département 8 de 100\$ si leur salaire est strictement inférieur à 5000.00\$, sinon de 50\$, en utilisant la clause de positionnement WHERE CURRENT OF.

Solution Implémentée

```

1  DECLARE
2      -- Declaration du curseur avec verrouillage explicite des
      lignes du dept 8
3      CURSOR c_emp_dept8 IS
4          SELECT id, salaire
5          FROM employees
6          WHERE departement_id = 8
7          FOR UPDATE OF salaire;
8
9      v_emp c_emp_dept8%ROWTYPE;
10     v_augmentation NUMBER;
11 BEGIN
12     OPEN c_emp_dept8;
13     LOOP
14         FETCH c_emp_dept8 INTO v_emp;
15         EXIT WHEN c_emp_dept8%NOTFOUND;
16
17         -- Application de la logique d'augmentation
            conditionnelle
18         IF v_emp.salaire < 5000.00 THEN
19             v_augmentation := 100.00;
20         ELSE
21             v_augmentation := 50.00;
22         END IF;
23
24         -- Modification precise via la ligne courante du
            curseur
25         UPDATE employees
26         SET salaire = salaire + v_augmentation
27         WHERE CURRENT OF c_emp_dept8;
28
29         DBMS_OUTPUT.PUT_LINE('Employ ID ' || v_emp.id || '
            revaloris de ' || v_augmentation || '$');
30     END LOOP;
31     CLOSE c_emp_dept8;
32
33     COMMIT; -- Validation definitive de la transaction
34 END;
35 /

```

Synthèse Didactique et Pièges Courants

L'attribut SQL%NOTFOUND vs SQL%FOUND

Au cours des manipulations de requêtes DML impératives (INSERT, UPDATE, DELETE), Oracle lève des variables d'état éphémères au sein du curseur implicite général baptisé SQL.

TABLE 7 – Comportement des attributs de transactions

Attribut	Si la ligne est modifiée/trouvée	Si aucune ligne n'est impactée
SQL%FOUND	TRUE	FALSE
SQL%NOTFOUND	FALSE	TRUE
SQL%ROWCOUNT	Entier ≥ 1 (Nombre de lignes)	0

Intérêt de la clause WHERE CURRENT OF

L'intégration du mot-clé WHERE CURRENT OF présente deux intérêts majeurs en ingénierie de bases de données :

- **Performances accrues** : Évite au moteur d'exécution d'Oracle de réévaluer une clause WHERE ou de refaire une recherche d'index secondaire coûteuse pour localiser la ligne. Le pointeur interne mémoire adresse directement le tuple physique (via son ROWID).
- **Sécurité transactionnelle** : Associé obligatoirement à la clause FOR UPDATE lors de la déclaration du curseur, il garantit qu'aucune autre session concurrente ne peut altérer ou modifier la ligne en cours de lecture pendant toute la durée de l'exécution du bloc.

Conclusion

Ce recueil d'exercices pratiques démontre l'agilité du langage PL/SQL pour orchestrer des opérations logiques locales et interagir efficacement avec les données relationnelles. La maîtrise des structures conditionnelles imbriquées, le suivi précis de l'impact des requêtes via l'analyse des attributs de curseurs (SQL%NOTFOUND), ainsi que le traitement ligne par ligne sécurisé par la clause WHERE CURRENT OF constituent le socle de développement indispensable pour concevoir des applications transactionnelles robustes et performantes sur un serveur de données Oracle.

TP 8 : Les Triggers PL/SQL avec application aux contraintes d'intégrité

Introduction aux Triggers PL/SQL

Un déclencheur (*trigger*) est un bloc de code PL/SQL nommé, stocké en base de données, qui s'exécute automatiquement (se « déclenche ») lorsqu'un événement spécifique survient (instructions DML : INSERT, UPDATE, DELETE). Ils sont indispensables pour appliquer des règles de gestion complexes qui ne peuvent pas être prises en charge par les contraintes d'intégrité déclaratives standard (CHECK, FOREIGN KEY, etc.).

Types de triggers

- **Trigger au niveau ligne (FOR EACH ROW)** : S'exécute autant de fois qu'il y a de lignes affectées par la requête DML.
- **Trigger au niveau commande (STATEMENT)** : S'exécute une seule fois par instruction, indépendamment du nombre de lignes modifiées.
- **Timing (BEFORE / AFTER)** : Détermine si le code s'exécute avant ou après la validation logique de l'action DML.

Pseudos-enregistrements : :NEW et :OLD

Dans un trigger de niveau ligne (FOR EACH ROW), le moteur Oracle fournit deux structures de données spéciales :

- **:NEW** : Contient les nouvelles valeurs en cours d'insertion ou de modification.
- **:OLD** : Contient les anciennes valeurs avant modification ou suppression.

Gestion des exceptions et transactions autonomes

- **RAISE_APPLICATION_ERROR** : Permet d'interrompre immédiatement l'exécution de la transaction DML courante et de renvoyer un message d'erreur personnalisé à l'application appelante (codes d'erreur entre -20000 et -20999).
- **PRAGMA AUTONOMOUS_TRANSACTION** : Permet à un trigger d'exécuter des opérations de validation autonomes (COMMIT/ROLLBACK) indépendantes de la transaction principale, ce qui s'avère indispensable pour contourner l'erreur de table mutante (*Mutating Table Error* - ORA-04091) lors de requêtes de lecture sur la table en cours de modification.

Contexte des exercices : Schéma de la Base de Données

Le cadre d'application est un système de gestion d'inscriptions universitaires structuré autour des tables physiques suivantes :

- **ETUDIANT** (id_etudiant, nom, prenom, date_inscription, moyenne)
- **MODULE** (code_module, intitule, effecMax, note_min)
- **PREREQUIS** (#module, #prerequis, note_min)
- **INSCRIPTION** (#id_etudiant, #code_module, date_inscription)
- **EXAMEN** (#id_etudiant, #code_module, date_examen, note)
- **PASSER** (#id_etudiant, #code_module, date_examen)

Implémentation des 9 contraintes par triggers

Règle 1 : Interdiction de modifier note_min dans prerequis

```

1 CREATE OR REPLACE TRIGGER trg_r1_note_min_prerequis
2 BEFORE UPDATE OF note_min ON prerequis
3 FOR EACH ROW
4 BEGIN
5     RAISE_APPLICATION_ERROR(-20001, 'R g l e 1 : Il est
        strictement interdit de modifier la note minimale d''un
        pr requis apr s sa cr ation.');
```

58 – Trigger de blocage sur modification de colonne

Règle 2 : Contrôle de effecMax lors d'une inscription

```

1 CREATE OR REPLACE TRIGGER trg_r2_effectif_max
2 BEFORE INSERT ON inscription
3 FOR EACH ROW
4 DECLARE
5     PRAGMA AUTONOMOUS_TRANSACTION;
6     v_actuel NUMBER;
7     v_max     NUMBER;
8 BEGIN
9     -- Extraction du nombre actuel d'inscrits
10    SELECT COUNT(*) INTO v_actuel FROM inscription WHERE
        code_module = :NEW.code_module;
11    -- Extraction du seuil maximal defini
12    SELECT effecMax INTO v_max FROM module WHERE code_module =
        :NEW.code_module;
13
14    IF v_actuel >= v_max THEN
15        RAISE_APPLICATION_ERROR(-20002, 'R g l e 2 : Inscription
            impossible. L''effectif maximal ( ' || v_max || ' )
            pour le module ' || :NEW.code_module || ' est
            atteint.');
```

59 – Trigger avec transaction autonome pour éviter la table mutante

Règle 3 : Création d'examen uniquement si le module a des inscrits

```
1 CREATE OR REPLACE TRIGGER trg_r3_examen_inscrits
2 BEFORE INSERT ON examen
3 FOR EACH ROW
4 DECLARE
5     PRAGMA AUTONOMOUS_TRANSACTION;
6     v_nbr_inscrits NUMBER;
7 BEGIN
8     SELECT COUNT(*) INTO v_nbr_inscrits FROM inscription WHERE
9         code_module = :NEW.code_module;
10
11     IF v_nbr_inscrits = 0 THEN
12         RAISE_APPLICATION_ERROR(-20003, 'R g le 3 : Impossible
13             de planifier un examen pour le module ' || :NEW.
14             code_module || ' car aucun tudiant n''y est
15             inscrit.');
```

60 – Vérification de la présence d'au moins un étudiant inscrit

Règle 4 : Date d'examen postérieure à l'inscription de l'étudiant

```

1 CREATE OR REPLACE TRIGGER trg_r4_date_coherence
2 BEFORE INSERT OR UPDATE OF date_examen ON examen
3 FOR EACH ROW
4 DECLARE
5     PRAGMA AUTONOMOUS_TRANSACTION;
6     v_date_insc DATE;
7 BEGIN
8     SELECT date_inscription INTO v_date_insc
9     FROM inscription
10    WHERE id_etudiant = :NEW.id_etudiant AND code_module = :NEW
        .code_module;
11
12    IF :NEW.date_examen < v_date_insc THEN
13        RAISE_APPLICATION_ERROR(-20004, 'R gle 4 : Coh rence
        de calendrier enfrente. La date de l''examen (' ||
        TO_CHAR(:NEW.date_examen, 'DD/MM/YYYY') || ') ne
        peut tre ant rieure la date d''inscription ('
        || TO_CHAR(v_date_insc, 'DD/MM/YYYY') || ').');
14    END IF;
15 EXCEPTION
16    WHEN NO_DATA_FOUND THEN
17        RAISE_APPLICATION_ERROR(-20004, 'R gle 4 : L''
        tudiant doit obligatoirement tre inscrit au
        module avant de passer un examen.');
```

18 END;

19 /

61 – Contrôle de cohérence chronologique des dates

Règle 5 : Détection de circuit dans la table prerequis

```

1 CREATE OR REPLACE TRIGGER trg_r5_anti_circuit
2 BEFORE INSERT OR UPDATE ON prerequis
3 FOR EACH ROW
4 DECLARE
5     PRAGMA AUTONOMOUS_TRANSACTION;
6     v_dummy NUMBER;
7 BEGIN
8     -- Recherche d'un chemin inverse existant dans le graphe
        des prerequis
9     SELECT 1 INTO v_dummy
```



```
10  FROM prerequis
11  WHERE module = :NEW.prerequis AND prerequis = :NEW.module
12  ROWNUM <= 1;
13
14  -- Si la requete retourne une ligne, un circuit direct est
    detecte
15  RAISE_APPLICATION_ERROR(-20005, 'R g l e 5 : D pendance
    cyclique d tect e. Le module ' || :NEW.prerequis || '
    est d j interd pendant de ' || :NEW.module);
16 EXCEPTION
17  WHEN NO_DATA_FOUND THEN
18      NULL; -- Absence de circuit detecte, l'operation est
    valide
19 END;
20 /
```

62 – Détection de boucle récursive (Ex: A nécessite B, et B nécessite A)

Règle 6 : Vérification des prérequis de note

```

1 CREATE OR REPLACE TRIGGER trg_r6_validation_prerequis
2 BEFORE INSERT ON inscription
3 FOR EACH ROW
4 DECLARE
5     PRAGMA AUTONOMOUS_TRANSACTION;
6     v_note_obtenue NUMBER;
7 BEGIN
8     -- Parcours de l'ensemble des prerequis imposes pour le
      module cible
9     FOR r IN (SELECT prerequis, note_min FROM prerequis WHERE
      module = :NEW.code_module) LOOP
10         BEGIN
11             -- Extraction de la note maximale obtenue par l'
              etudiant pour ce prerequis
12             SELECT MAX(note) INTO v_note_obtenue
13             FROM examen
14             WHERE id_etudiant = :NEW.id_etudiant AND
              code_module = r.prerequis;
15
16             IF v_note_obtenue IS NULL OR v_note_obtenue < r.
              note_min THEN
17                 RAISE_APPLICATION_ERROR(-20006, 'R gle 6 :
              Inscription rejet e. La note obtenue au
              pr requis ' || r.prerequis || ' est
              insuffisante (' || NVL(v_note_obtenue,0) ||
              '/' || r.note_min || ').');
18             END IF;
19         EXCEPTION
20             WHEN NO_DATA_FOUND THEN
21                 RAISE_APPLICATION_ERROR(-20006, 'R gle 6 : Le
              module pr requis ' || r.prerequis || ' n''a
              jamais t valid par cet tudiant .');
22         END;
23     END LOOP;
24 END;
25 /

```

63 – Validation des notes obtenues aux modules prérequis constitutifs

Règle 7 : Vérification de la moyenne minimale requise du module

```

1 CREATE OR REPLACE TRIGGER trg_r7_moyenne_module
2 BEFORE INSERT ON inscription
3 FOR EACH ROW
4 DECLARE
5     PRAGMA AUTONOMOUS_TRANSACTION;
6     v_moy_etudiant NUMBER;
7     v_note_min_mod NUMBER;
8 BEGIN
9     SELECT moyenne INTO v_moy_etudiant FROM etudiant WHERE
        id_etudiant = :NEW.id_etudiant;
10    SELECT note_min INTO v_note_min_mod FROM module WHERE
        code_module = :NEW.code_module;
11
12    IF v_moy_etudiant < v_note_min_mod THEN
13        RAISE_APPLICATION_ERROR(-20007, 'R g le 7 : Acc s
            refus . La moyenne g n rale de l'' tudiant (' ||
            v_moy_etudiant || ') est infrieure au niveau d''
            entr e requis pour ce module (' || v_note_min_mod
            || ').');
14    END IF;
15 END;
16 /

```

64 – Comparaison de la moyenne générale avec le seuil du module

Règle 8 : Calcul et mise à jour automatique de la moyenne de l'étudiant

```

1 CREATE OR REPLACE TRIGGER trg_r8_calcul_moyenne
2 AFTER INSERT OR UPDATE ON examen
3 FOR EACH ROW
4 DECLARE
5     PRAGMA AUTONOMOUS_TRANSACTION;
6     v_nouvelle_moyenne NUMBER;
7 BEGIN
8     -- Calcul de la moyenne arithmétique globale de l'étudiant
       concerne
9     SELECT AVG(note) INTO v_nouvelle_moyenne
10    FROM examen
11   WHERE id_etudiant = :NEW.id_etudiant;
12
13    -- Repercussion physique immédiate sur la fiche de l'
       etudiant
14    UPDATE etudiant
15    SET moyenne = v_nouvelle_moyenne
16    WHERE id_etudiant = :NEW.id_etudiant;
17
18    COMMIT; -- Validation de la transaction autonome d'
       actualisation
19 END;
20 /

```

65 – Mise à jour de la table etudiant suite à l'introduction d'une note

Règle 9 : Table d'archive pour les désinscriptions

```

1 -- 1. Creation prealable de la structure d'archivage
2 -- CREATE TABLE archive_inscription (
3 --     id_etudiant NUMBER,
4 --     code_module VARCHAR2(10),
5 --     date_desinscription DATE,
6 --     operateur VARCHAR2(50)
7 -- );
8
9 CREATE OR REPLACE TRIGGER trg_r9_audit_desinscription
10 AFTER DELETE ON inscription
11 FOR EACH ROW
12 BEGIN

```

```
13      INSERT INTO archive_inscription (id_etudiant, code_module,
14      date_desinscription, operateur)
15      VALUES (:OLD.id_etudiant, :OLD.code_module, SYSDATE, USER);
16  END;
17  /
```

66 – Création de la table d’audit et écriture de traçabilité

Conclusion

L’implémentation de ces 9 déclencheurs met en évidence la flexibilité et la robustesse des triggers PL/SQL pour administrer des contraintes complexes au sein d’un système d’information. Grâce à l’utilisation conjointe des variables de contexte (‘:NEW’/‘:OLD’) et des transactions autonomes (‘PRAGMA AUTONOMOUS_TRANSACTION’), le serveur de base de données Oracle

Cette centralisation du code métier directement sur le serveur garantit l’intégrité absolue de la base, indépendamment des applications clientes qui s’y connectent.